

10

Ensemble Techniques

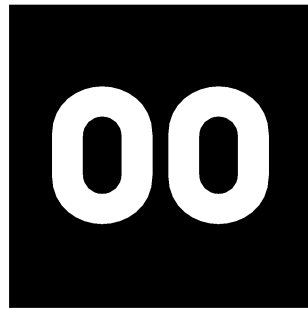
Data Mining II | MGI 2526

May 5 and 6, 2026

Carina Albuquerque

calbuquerque@novaims.unl.pt

Agenda



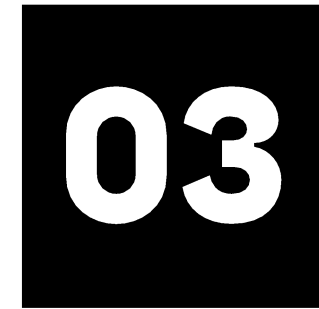
**Ensemble
Techniques**



**Bagging
Ensembles**



**Boosting
Ensembles**



**Stacking
Ensemble**



00.

Ensemble Techniques

0. Ensemble Techniques

Introduction

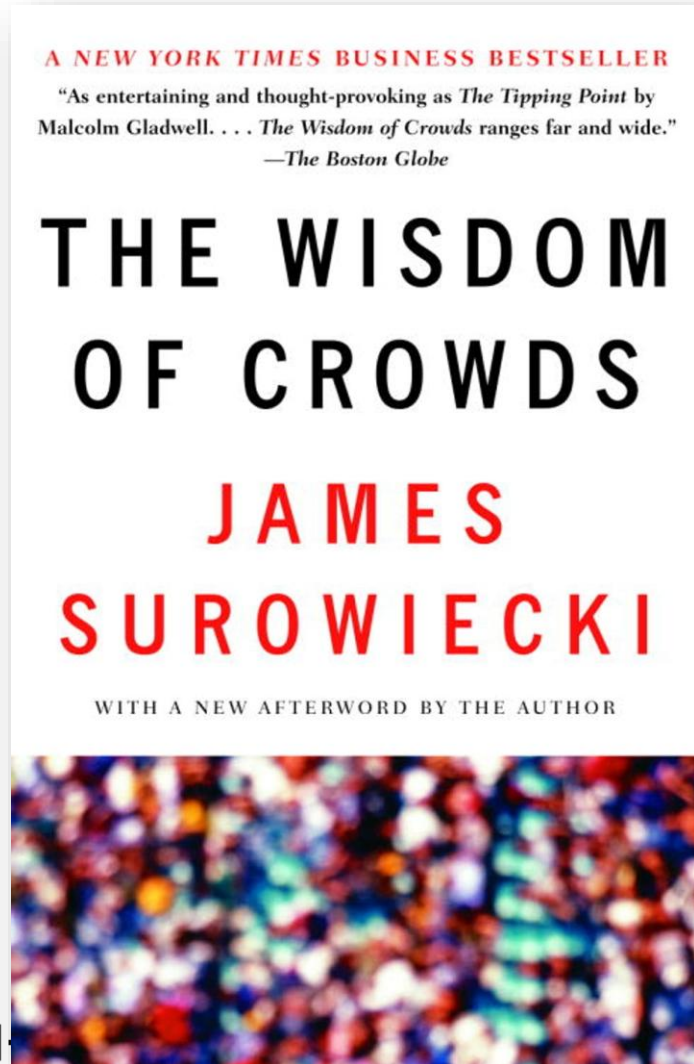


**What are
Ensemble
Techniques?**

Models that **combine** the decisions from **multiple models** to improve overall performance

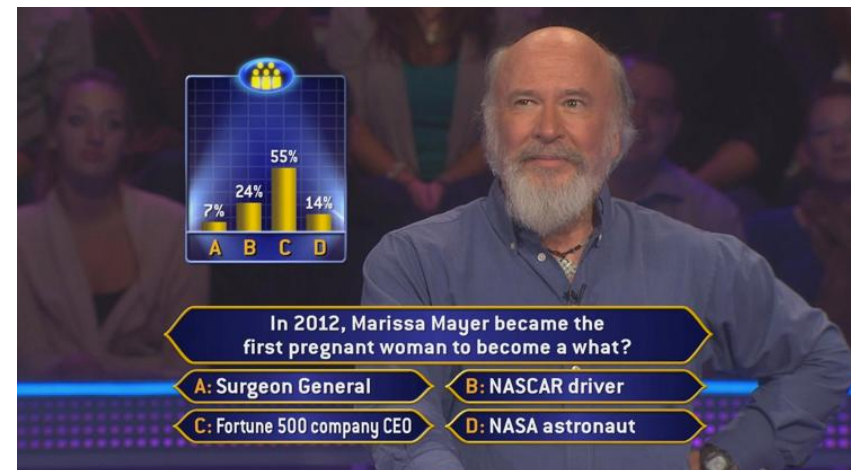
0. Ensemble Techniques

Introduction



“**Large groups of people are smarter than an elite few**, no matter how brilliant—better at solving problems, fostering innovation, coming to wise decisions, even predicting the future.”

“Diversity and independence are important because **the best collective decisions are the product of disagreement** and contest, not consensus or compromise.”



0. Ensemble Techniques

Introduction

The basic idea:

- Instead of relying on a single model to make predictions, ensemble techniques use the collective wisdom of several models to achieve better results
- By aggregating different models, ensembles can mitigate the weaknesses of individual models and exploit their complementary strengths.

In general...

The advantage: Usually improve the predictive performance, being able to handle complex relationships

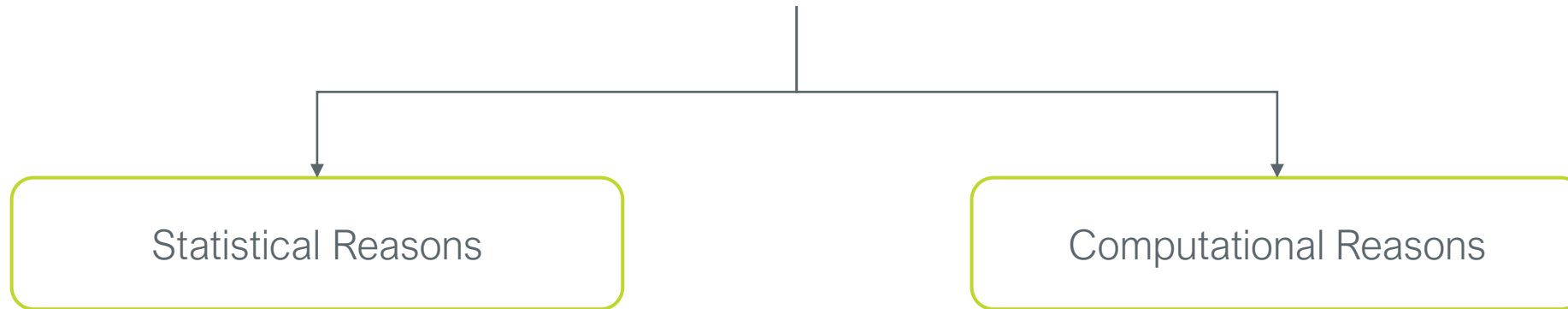
The disadvantage: Usually leads to an increased computational cost and some techniques lead to lack of interpretability

0.1. Ensemble Techniques

Why do Ensembles work?

Why do Ensembles work?

(Dietterich, 2000)



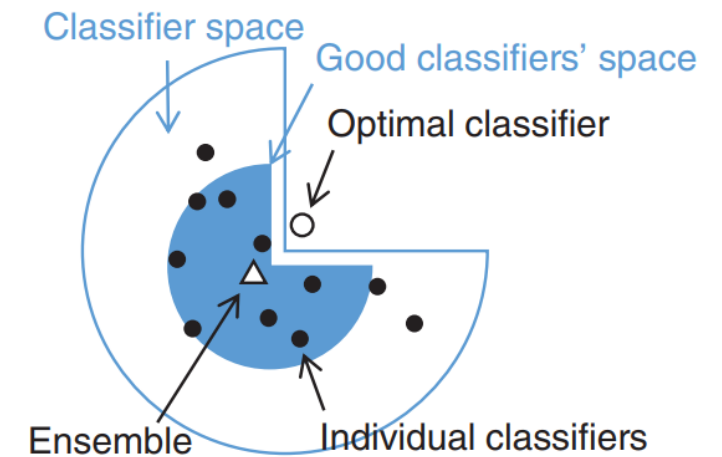
Dietterich, T. G. (2000, June). Ensemble methods in machine learning. In *International workshop on multiple classifier systems* (pp. 1-15). Berlin, Heidelberg: Springer Berlin Heidelberg.

0.1. Ensemble Techniques

Why do Ensembles work?

Statistical Reasons

1. Learning algorithms look for a solution to a problem on a landscape of hypothesis H
 - Given finite amounts of data, **multiple hypothesis may be equally “accurate” to address a problem.**
 - Those hypothesis can capture different aspects of patterns but yield similar predictive performance.
2. The **average of different potentially accurate hypothesis may result in a better approximation** than any individual model
 - Instead of relying on a single model, combining these potentially accurate hypotheses can result in a better approximation of the true



0.1. Ensemble Techniques

Why do Ensembles work?

Computational Reasons

1. Imperfect training algorithm

Algorithms generally tend to local optima (i.e. no guarantee of “the best” solution).

- The combination of the outputs of several suboptimal classifiers may lead to a better overall classifier

2. Divide and conquer approach

By splitting a complex problem in smaller one, each classifier is trained to handle the smaller problems

0.1. Ensemble Techniques

Why do Ensembles work?

Computational Reasons

3. Too much data

Instead of using all data in a single training algorithm, allows training of multiple models on smaller subsets of data

4. Small Data

Resampling is a good approach when the sample size is small

5. Different data natures

Mixed data types collected from heterogenous sources can be trained individually and then combined on ensembles

0.1. Ensemble Techniques

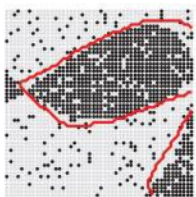
Why do Ensembles work?

A practical problem...

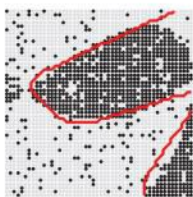
We want to apply a Multi-Layer perceptron classifier on an image to distinguish between black dots and gray dots

1. Create and train nine instances of the MLP (10 hidden neurons, weights randomly initialized)
 - We add some noise on the training data to increase the model's robustness to variations
2. Measure their performance by measuring the error rate

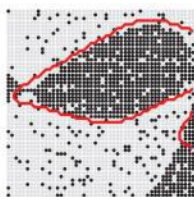
Results (Error Rate)



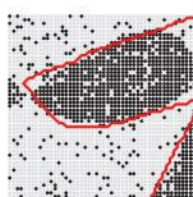
(a) 7.44%



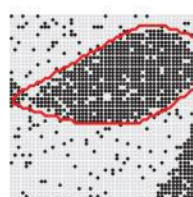
(b) 8.32%



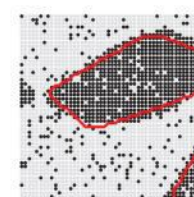
(c) 9.88%



(d) 7.64%



(e) 9.72%



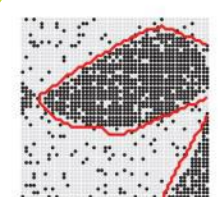
(f) 7.76%



(g) 9.64%



(h) 10.20%



(i) 6.44%

This is the model with the lowest error

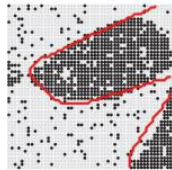
0.1. Ensemble Techniques

Why do Ensembles work?

Results (Error Rate)



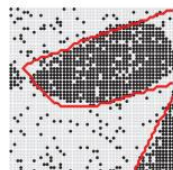
(a) 7.44%



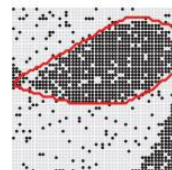
(b) 8.32%



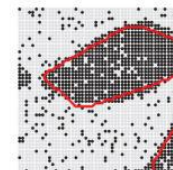
(c) 9.88%



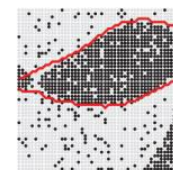
(d) 7.64%



(e) 9.72%



(f) 7.76%



(g) 9.64%



(h) 10.20%

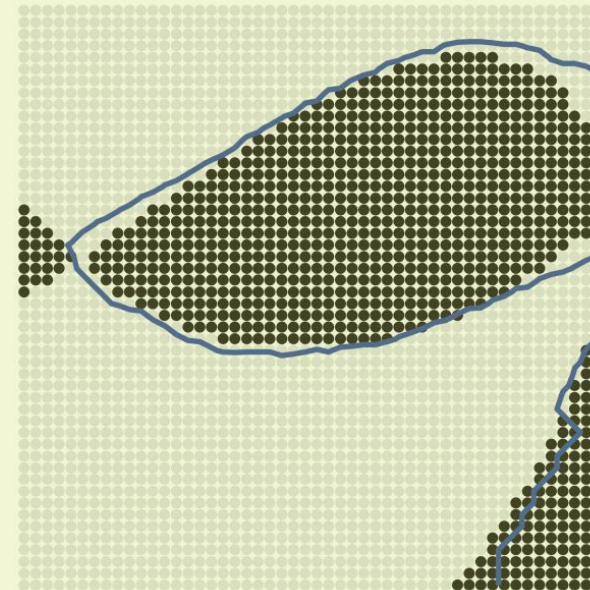


(i) 6.44%

Mean of the individual errors: $(7.44 + 8.32 + \dots + 6.44) / 9 = 8.56\%$

How to
minimize
this error?

For each pixel,
I am going to consider
majority voting!

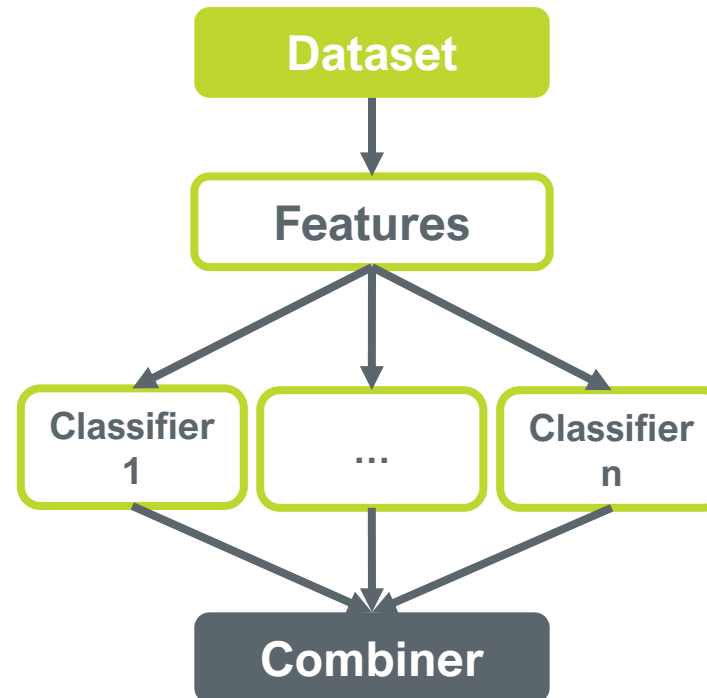


Error Rate: 5.84%

0.2. Ensemble Techniques

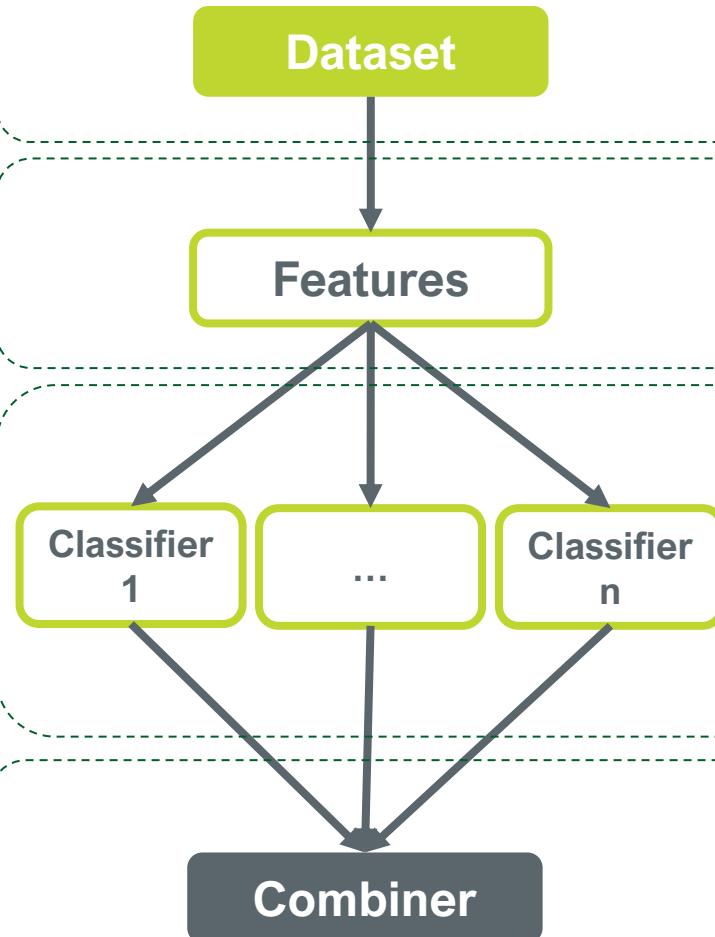
Setting up an Ensemble

How to setup an ensemble?



0.2. Ensemble Techniques

Setting up an Ensemble



- How can we manipulate the data submitted for training to the base classifier in order to ensure both high diversity and high individual accuracy?
- Shall we use all features or use a customized subset for each classifier?
- How do we select / extract such subsets?
- Do we use the same or different classifiers?
- What base classifier is the best? Decision Trees, Neural Networks, ...
- How many classifiers are needed?
- Should the classifiers be trained in parallel or sequentially?
- How are the individual inputs combined?

0.3. Ensemble Techniques

Bias and Variance in Ensemble Techniques

The goal...

- Combine base estimators to improve generalizability over a single estimator

Ensemble methods are meta-algorithms that combine ML techniques into a single predictive model to:

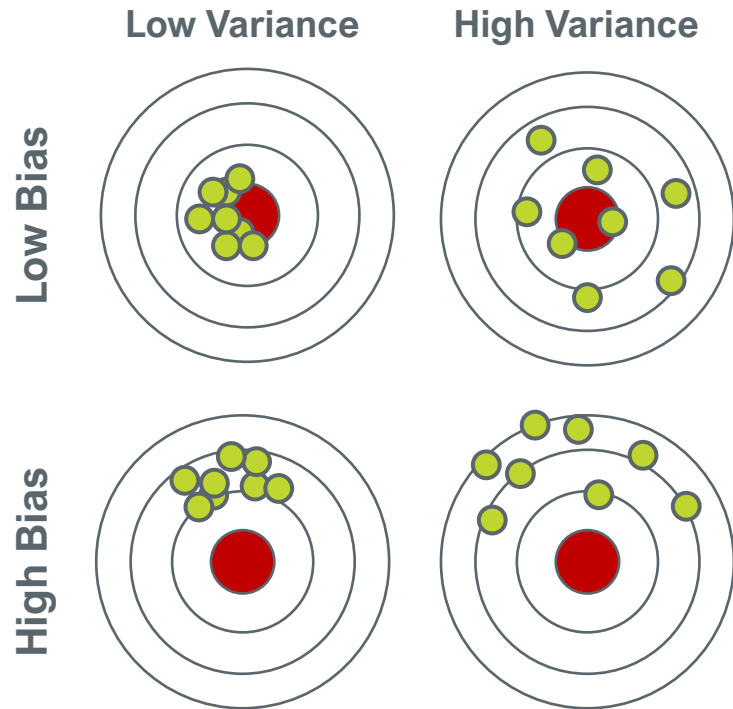
- Decrease **variance** (Bagging Techniques)
- Decrease **bias** (Boosting Techniques)
- Improve predictive **performance** (Stacking Techniques)

What is a meta-algorithm?

- A higher-level algorithm that operates on the outputs of multiple base learners to make predictions or decisions.

0.3. Ensemble Techniques

Bias and Variance in Ensemble Techniques



- **Bias** measures how far off the predictions of a model are from the true values on average
- **Variance** measures the variability of model predictions for a given dataset.

A high bias model tends to **oversimplify** the data and miss important patterns, with high error on train and on test → **underfitting**

A high variance model is highly sensitive to small changes in the data by capturing noise instead of patterns → **overfitting**

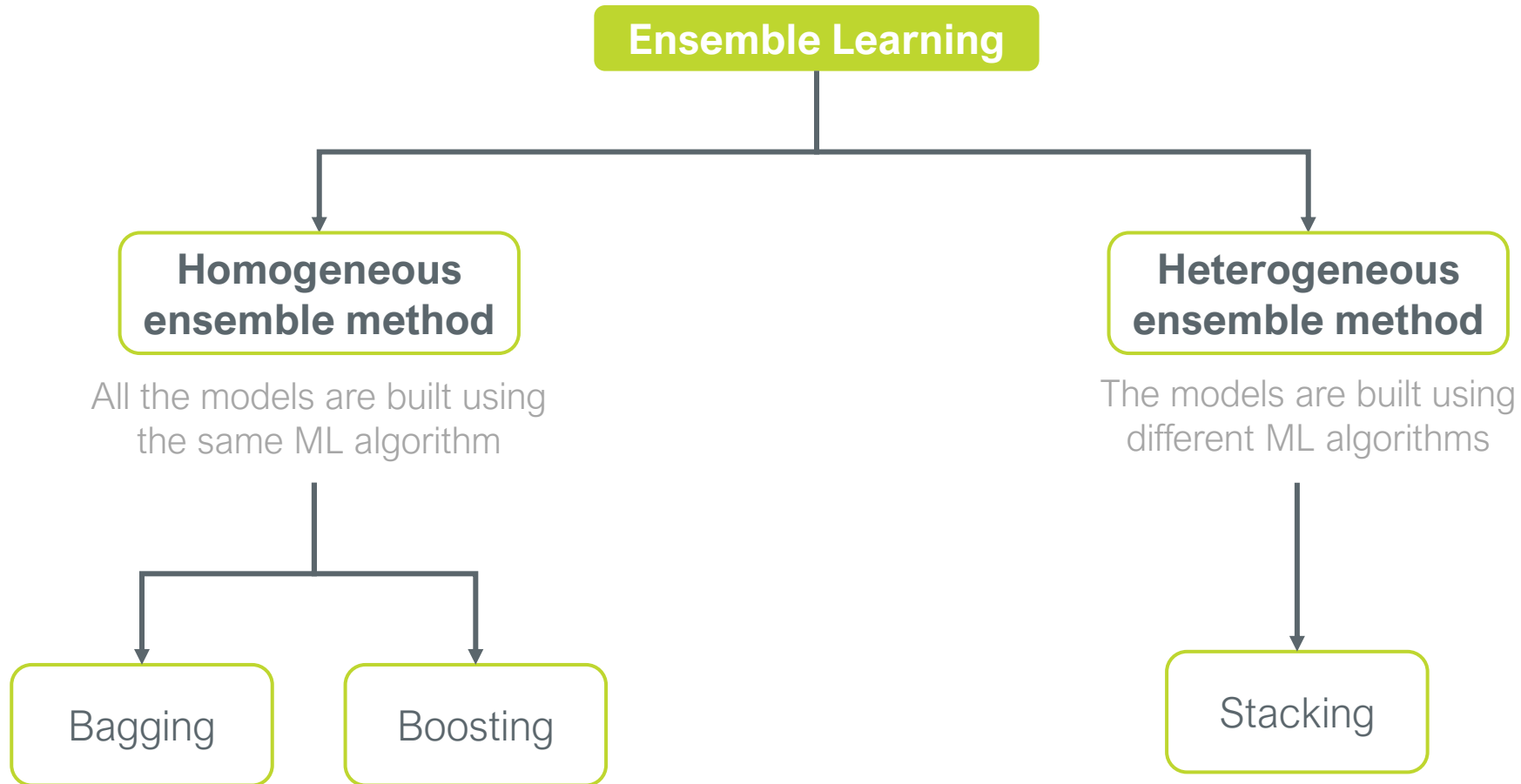
Bias Variance Trade-off

$$\text{Error} = \text{bias}^2 + \text{variance} + \text{irreducible error}$$

- Increase bias reduces variance and vice-versa
- The best model is where the error is reduced
- We need to have a compromise between bias and variance

0.4. Ensemble Techniques

Homogeneous and Heterogeneous Ensemble Techniques





01.

Bagging Ensembles

1. Bagging Ensemble

Introduction

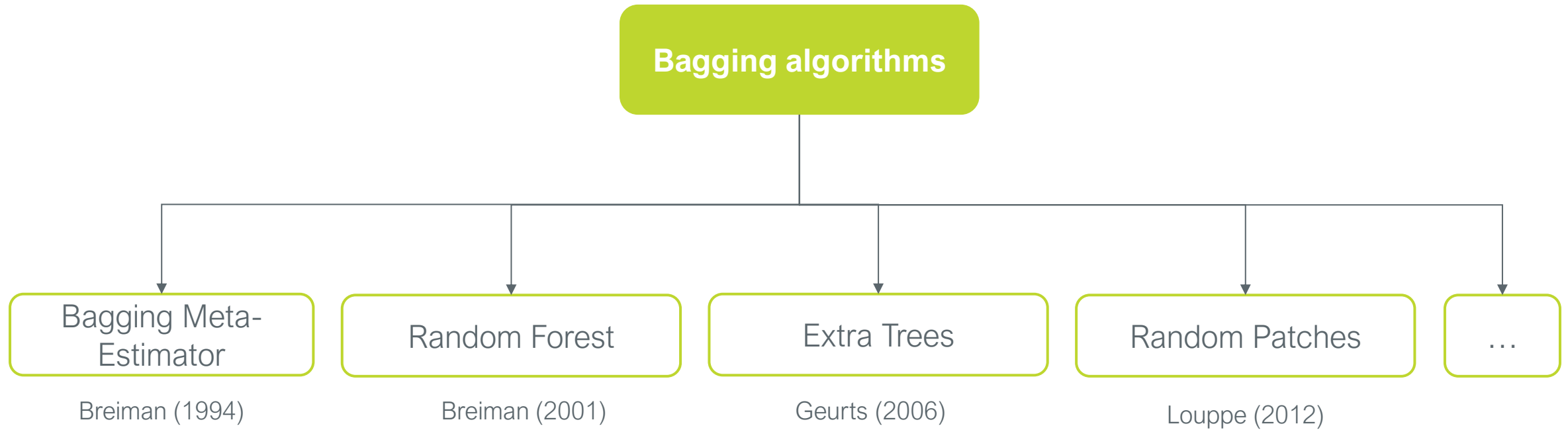


**What is
bagging?**

Ensemble learning method that trains **parallelly multiple instances of the same algorithm**, commonly used to reduce variance within a noisy dataset

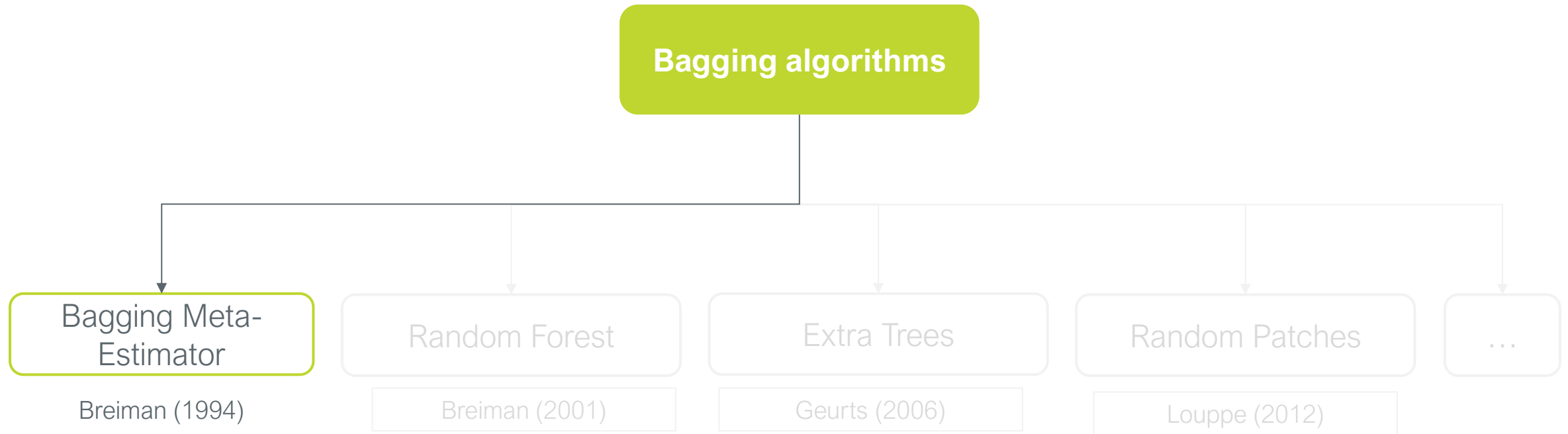
1. Bagging Ensemble

Introduction



1.1. Bagging Meta-Estimator (AKA Bagging)

Introduction



1.1.1. Bagging Meta-Estimator (AKA Bagging)

Introduction

What is Bagging (Bootstrap Aggregating)?

- Simple idea: The ensemble is made of classifiers built on **bootstrap** replicates of the training set. The classifier outputs are combined by the majority vote.

The diversity is created by using different training sets

- **Sample with replacement** from the original training set
- To make use of the variations of the training set, **the base learner should be unstable**
 - Small changes in the training set should lead to large changes in the classifier output (otherwise the final ensemble will be a collection of almost identical classifiers → no improvement)

1.1.1. Bagging Meta-Estimator (AKA Bagging)

Introduction

What is the number of samples (B) for each training bootstrapped set?

- Usually $B < N$ (rule of thumb $\rightarrow$ 60%), where N is the total number of samples in the training set

How large should a bagging ensemble be?

- Breiman found that 25-50 base learners are sufficient to get the ensemble error to level off (independently of the classifier model used and of the application domain).

Can bagging be used with any type of base model?

- Yes, bagging can be used with any type of base model, including decision trees, support vector machines, neural networks, and more.
- However, it is most commonly used with decision trees due to their tendency to overfit to the training data, to the fast training and since small variations on data can lead to completely different trees.

1.1.2. Bagging Meta-Estimator (AKA Bagging)

The process

How does it work?

- Train multiple instances of the same algorithm using **bootstrap** replicas of the training set
- The training for each model is performed in **parallel** (trained in different processors if needed)
- The predictions of each individual are **combined** to obtain a final prediction
- The main focus is on reducing **variance**

What is bootstrapping?

- a fundamental component that allows for the creation of diverse base models by training them on slightly different subsets of the data

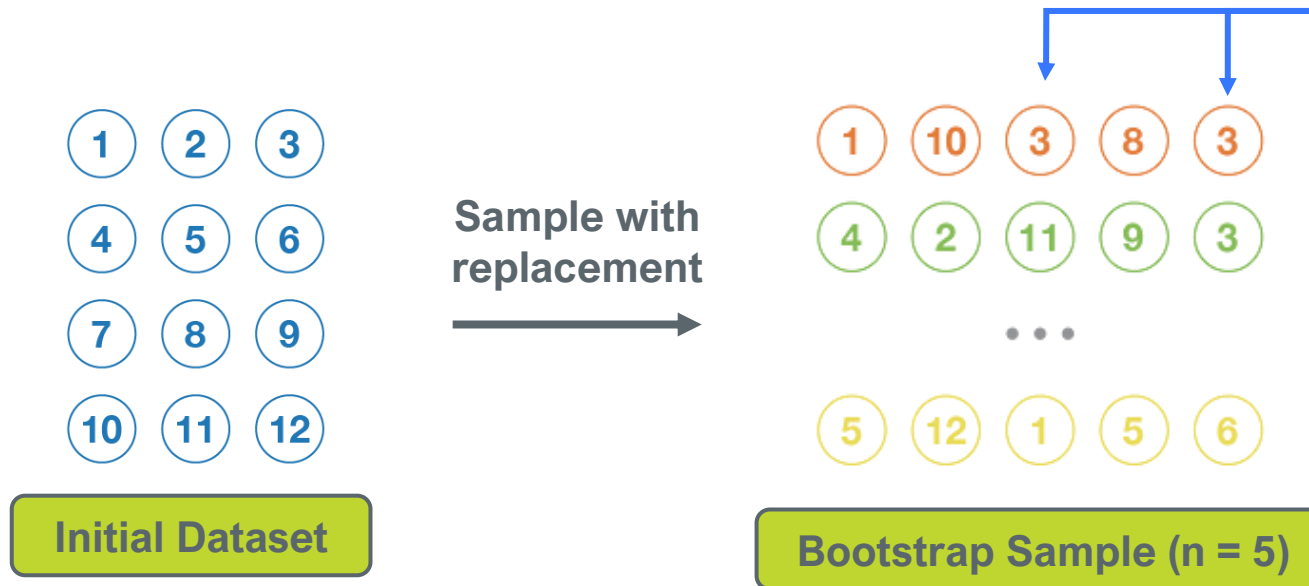
1.1.2. Bagging Meta-Estimator (AKA Bagging)

The process

What is bootstrapping?

The steps:

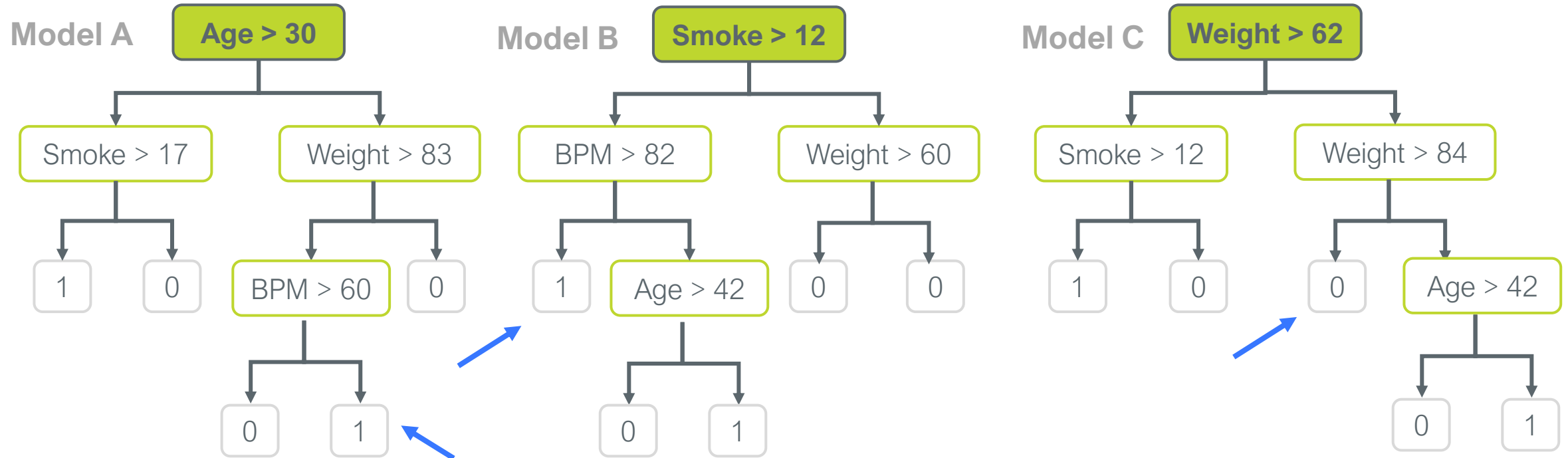
1. Start with the full training data set with N observations
2. Determine the size of the bootstrap sample you want to create (good starting point is 60% of N)
3. Randomly select observations from the original dataset with replacement to create a bootstrap sample.



- Each observation has an equal probability of being selected at each iteration.
- The same observation can be selected multiple times.

1.1.3. Bagging Meta-Estimator (AKA Bagging)

New predictions on Classification



New observation:

Age	Smoke	Weight	BPM	Disease
34	12	67	72	?

Model A

1

Model B

1

Model C

0

Majority Voting: **1**

1.1.3. Bagging Meta-Estimator (AKA Bagging)

New predictions on Classification

In classification we have two ways of doing the voting...

Hard voting

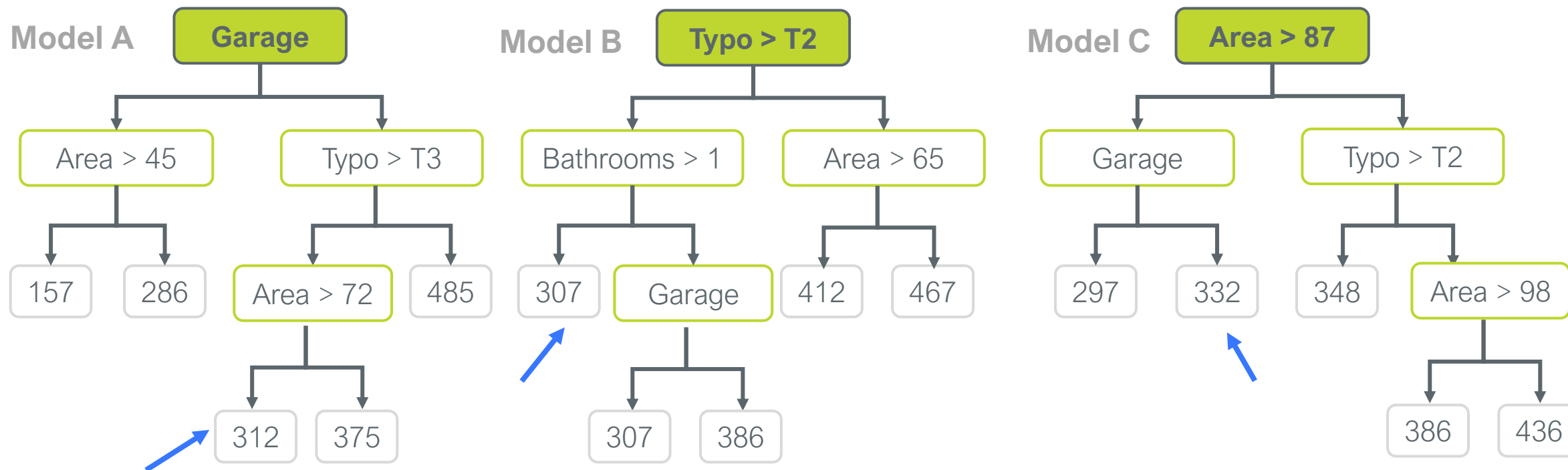
- It takes a simple majority vote to decide the final prediction, based on the most frequent class predicted by individual models.

Soft Voting

- It considers the probability scores of each class predicted by individual models and averages them to produce a more refined final prediction.
- Soft voting takes into account the confidence or certainty of each model's prediction, rather than just the class labels themselves.

1.1.3. Bagging Meta-Estimator (AKA Bagging)

New predictions on Regression



New observation:

Garage	Typo	Area	Bathrooms	Price
Yes	T2	67	1	?

Model A

312

Model B

307

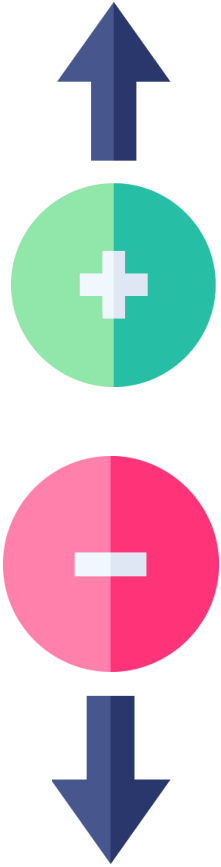
Model C

332

Weighted Average: **317K**

1.1.4. Bagging Meta-Estimator (AKA Bagging)

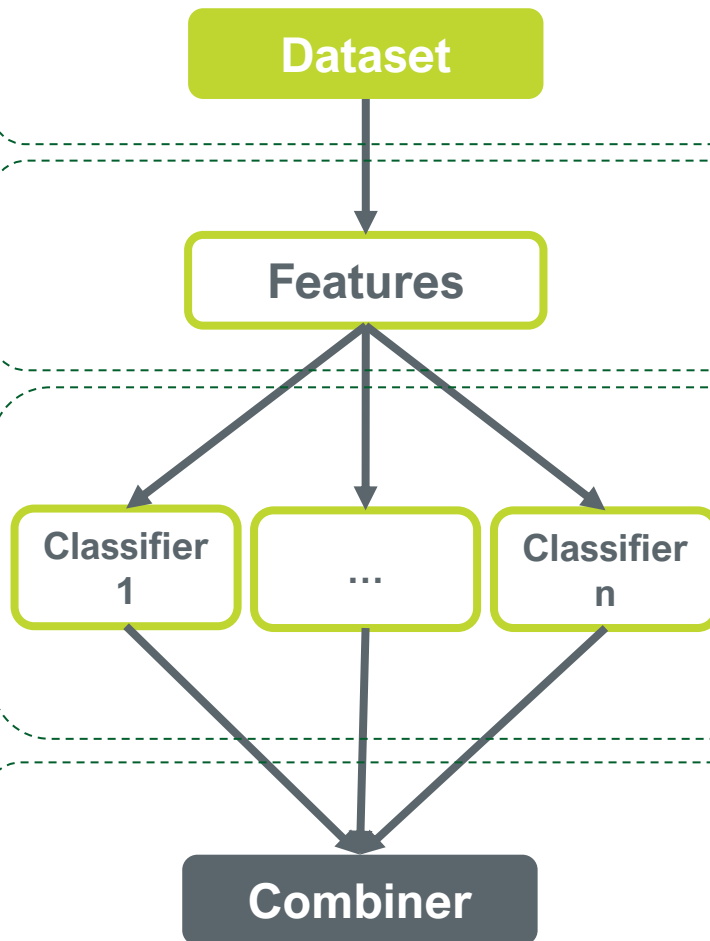
Advantages and Disadvantages



- Combine weak learners to outdo a strong learner
- **Reduces variance**
- Works with **any type of model** (but the main focus is on DT)
- Easy to **parallelize**
- **Less interpretable** than a single DT
- As each tree works as a fully functional DT, its **trees may become similar**
- Tends to **generate correlated trees**
 - Averaging correlated predictions usually does not reduce variance significantly

1.1. Bagging Meta-Estimator (AKA Bagging)

RECAP



- Trains multiple base models on **bootstrapped subsets** (may contain duplicates and variations of the original training data) providing diversity to the base models.
- Uses typically **all features**.
- Several algorithms can be used as learners, but typically, preference is given to **algorithms highly sensitive to small variations** in the training data.
- Training individual models in bagging can be **parallelized**
- Breiman found that **25-50 classifiers** are enough
- **Regression problem:** average of the outputs of individual models
- **Classification problem:** Voting (hard-voting or soft voting)

1.2. Bagging Meta-Estimator (AKA Bagging)

Random Forest



1.2.1. Random Forest

Introduction

What are Random Forests?

- Similar to Bagging Decision Trees
 - It also bootstraps instances to create a training set for each tree
 - Each tree is grown to the maximum extend possible

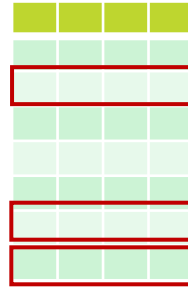
So... what is different?

1. The size of the bootstrap samples is usually as the same size of the training set
2. Additionally, it also randomly subset the features to include in each individual tree
 - The number of features to subset is a tunable parameter
3. It only uses Decision Trees as learners

1.2.1. Random Forest

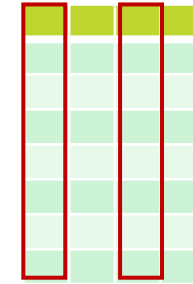
Introduction

The randomness in Random Forest



Bootstrapping

Choosing a random training set
(using bootstrap) for each base
learner (a decision tree)



Feature Selection

At each node of the DT, a feature should be
selected to split and grow the tree to the next level.
This feature is chosen from a random subset of
features of the training data.

1.2.2. Random Forest

The process

1. Create a bootstrapped dataset with n observations

My data...

ID	Fever	Cough	Fatigue	Weight	Disease
1	No	No	No	62	No
2	Yes	Yes	Yes	85	Yes
3	Yes	Yes	No	91	No
4	Yes	No	Yes	73	Yes



Bootstrapped dataset

ID	Fever	Cough	Fatigue	Weight	Disease
2	Yes	Yes	Yes	85	Yes
1	No	No	No	62	No
3	Yes	Yes	No	91	No
3	Yes	Yes	No	91	No

1.2.2. Random Forest

The process

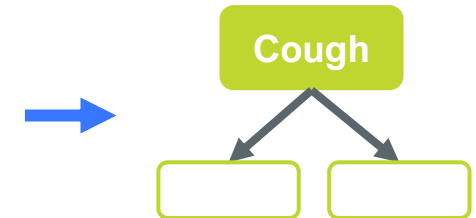
2. Start creating a decision tree using the bootstrapped dataset, but use only a random subset of variables at the first step

Bootstrapped dataset

ID	Fever	Cough	Fatigue	Weight	Disease
2	Yes	Yes	Yes	85	Yes
1	No	No	No	62	Yes
3	Yes	Yes	No	91	No
3	Yes	Yes	No	91	No

→
Select 2 random
variables (tunable
hyperparameter)

ID	Cough	Fatigue
2	Yes	Yes
1	No	No
3	Yes	No
3	Yes	No



1.2.2. Random Forest

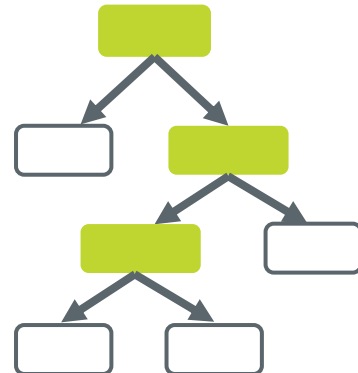
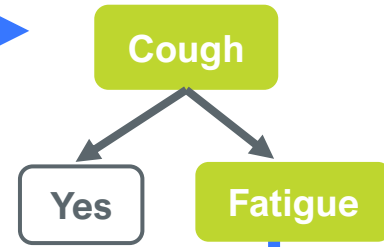
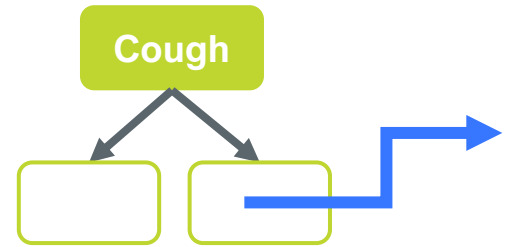
The process

3. For each split, consider a random subset of variables and choose the variable with the highest discriminative power.

Bootstrapped dataset

ID	Fever	Cough	Fatigue	Weight	Disease
2	Yes	Yes	Yes	85	Yes
1	No	No	No	62	Yes
3	Yes	Yes	No	91	No
3	Yes	Yes	No	91	No

ID	Fever	Fatigue
2	Yes	Yes
1	No	No
3	Yes	No
3	Yes	No



...

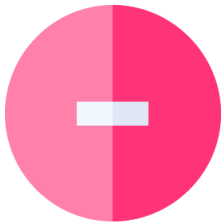
From the bootstrapped dataset get two random variables and select the one that better discriminates the target

1.2.3. Random Forest

Advantages and disadvantages



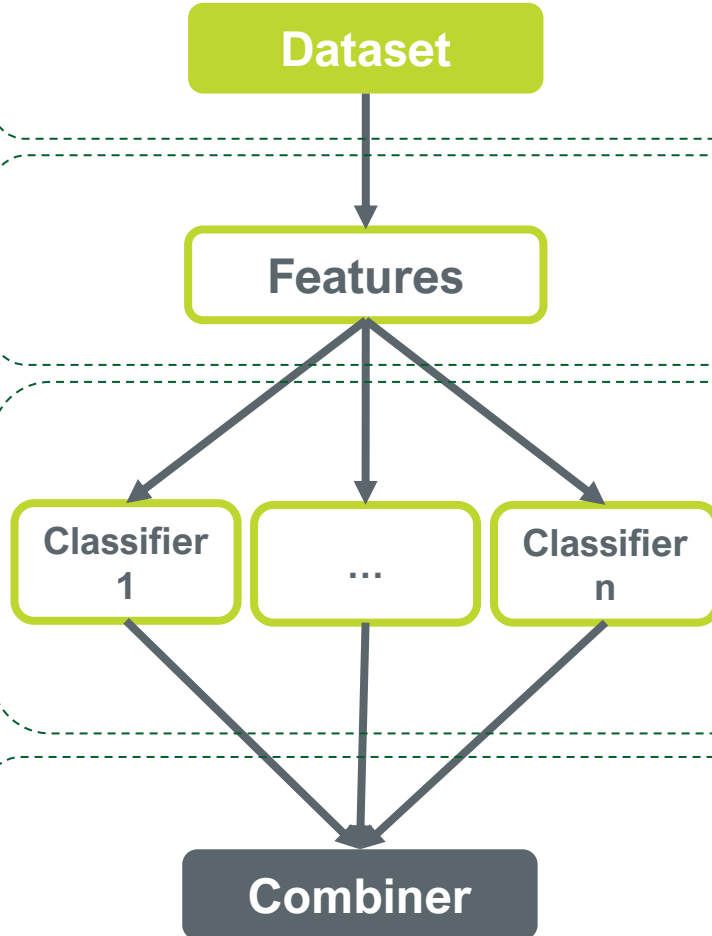
- Run efficiently on **large data sets**
- Can deal well with a **large number of features**
- Easy to **parallelize**
- Since it selects one feature at each split from a subset of features (and not from all features), a single learner trains faster than a Decision Tree



- **Less interpretable** than a single DT
- Can be **computationally intensive** and require more resources compared to simpler models
- Have **several hyperparameters** that need to be tuned for optimal performance
- Require storing multiple decision trees, which can consume significant **memory resources**

1.2. Random Forest

RECAP



- Trains multiple base models on **bootstrapped subsets**, similar to bagging
- Each split considers **only a random subset of features**, introducing randomness and promoting tree decorrelation for a diverse ensemble.
- Fully grown-up **decision trees** are used
- Training individual trees in a random forest can be **parallelized**
- It is common to start with a reasonable number of trees, such as **100**, and then tune this hyperparameter.

Similar to bagging:

- **Regression problem:** Average of the outputs of individual models
- **Classification problem:** Voting (hard-voting or soft voting)



02.

Boosting Ensembles

2. Boosting Ensemble

Introduction



**What is
boosting?**

Ensemble learning method that combines a set of weak learners into a strong learner to minimize training errors

sequentially

2. Boosting Ensemble

Introduction

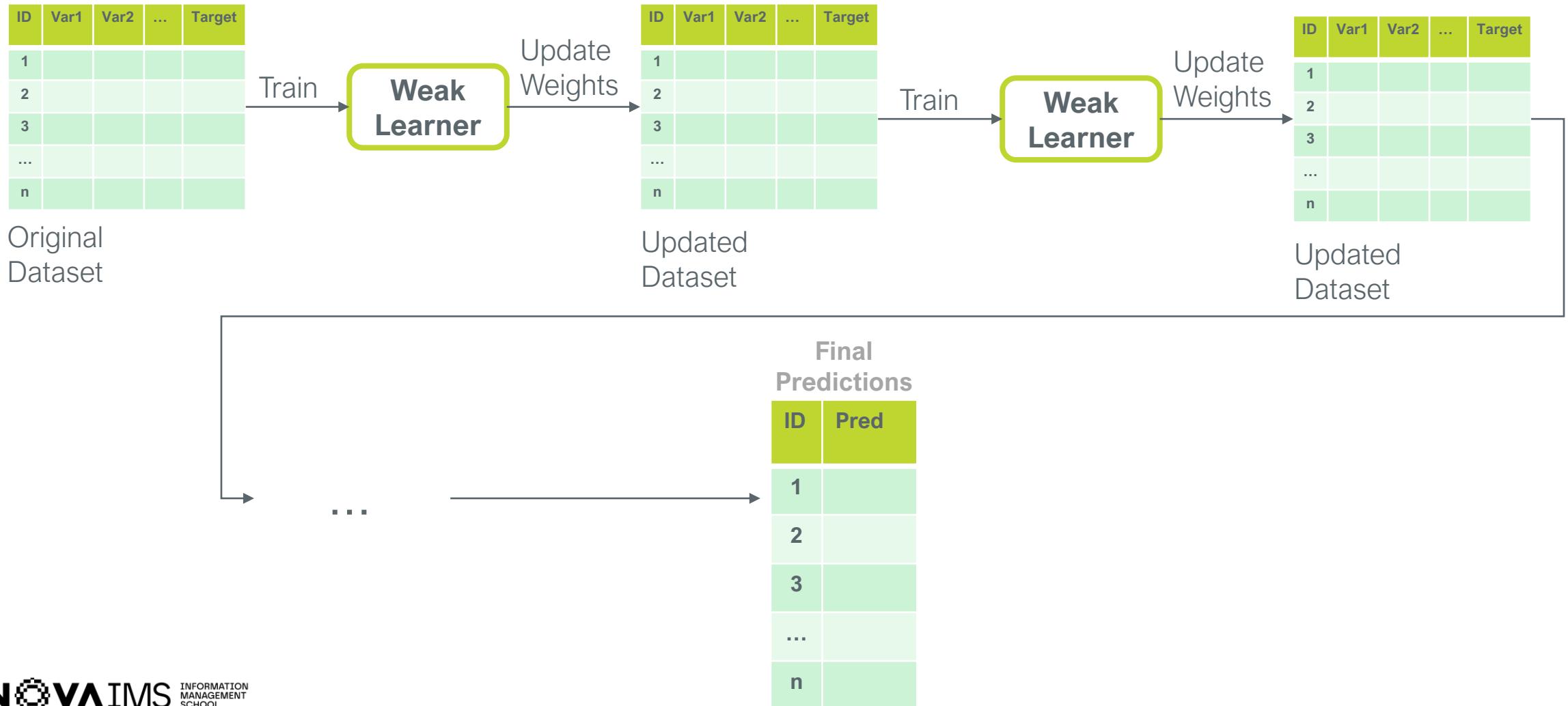
The idea behind...

A **weak algorithm** that performs just slightly better than random guessing, **can be boosted** into a **strong learning algorithm** in a set of interactive trainings.

- There are different versions of boosting algorithms proposed and implemented
 - The **main difference** among them is **their method of weighting training data points** and hypothesis.
- Mainly focused on **reducing bias**
- Can't be done in parallel (the process is done **sequentially**)

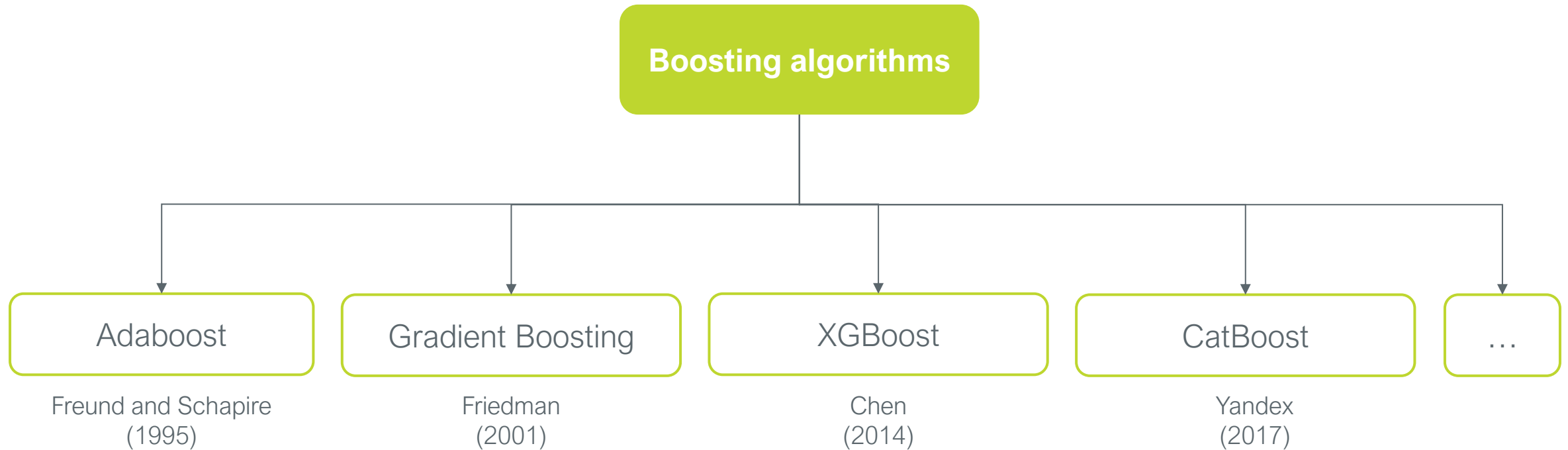
2. Boosting Ensemble

Introduction



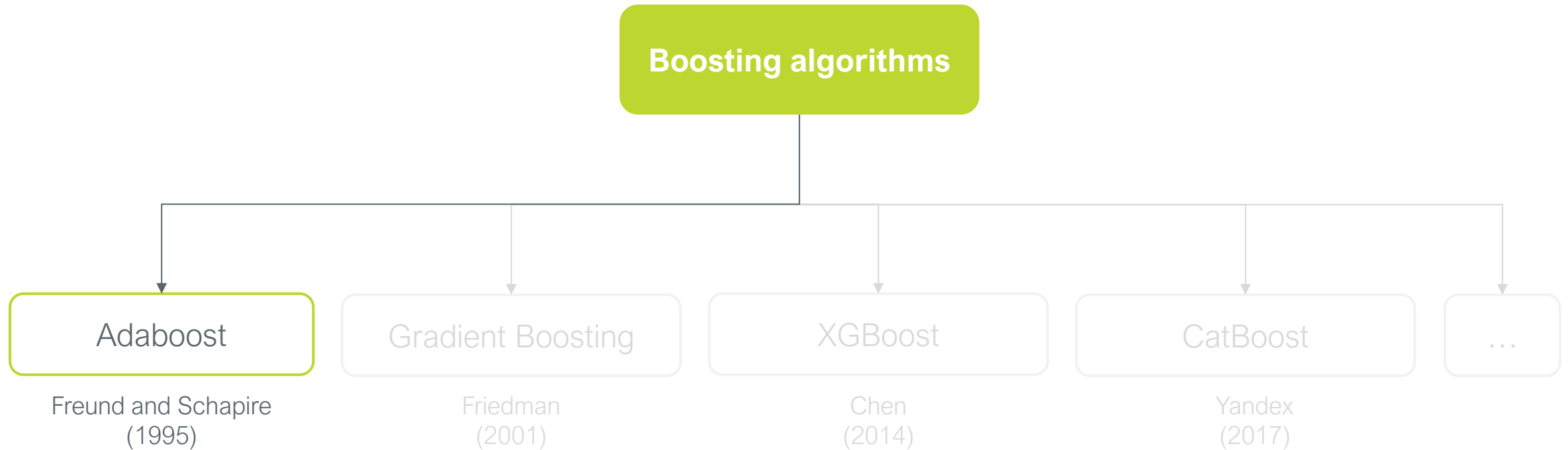
2. Boosting Ensemble

Introduction



2.1. Adaboost

Introduction



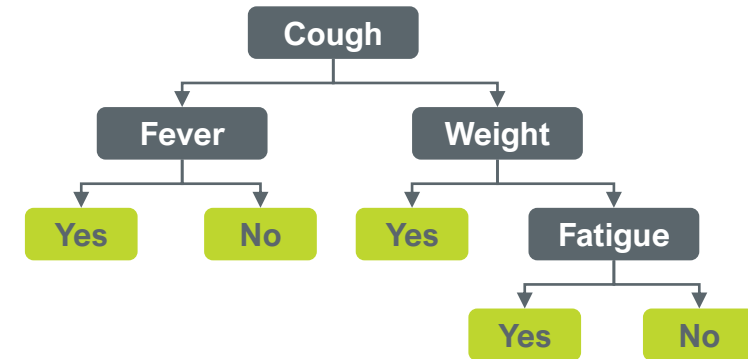
2.1.1. Adaboost

Differences between a Random Forest and Adaboost

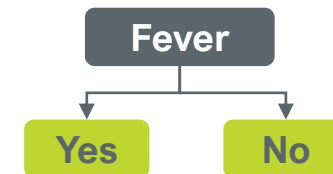
1. In the structure of the decision tree...

- In a **Random Forest**, each tree is a full-grown-up tree
 - Some trees might be bigger than others depending on the data, but there is no predetermined maximum depth
- In **Adaboost**, each tree is just the root node and two leaves, which are called stumps

Tree in a Random Forest



Tree in Adaboost (Stump)



Weak Learner

2.1.1. Adaboost

Differences between a Random Forest and Adaboost

2. In the weight of each tree on the final prediction

- In a **Random Forest**, each tree has an equal vote on the final classification
- In **Adaboost**, some stumps have more weight on the final prediction than others

3. In the independence of the trees

- In **Random Forest**, each decision tree is made independently of the others
- In **Adaboost**, order is important
 - The errors that the first stump makes, influence how the second stump is made, and so on...

2.1.2. Adaboost

The process

1st Step: Assign a sample weight for each sample

ID	Fever	Fatigue	Weight	Disease	Sample Weight
1	Yes	Yes	92	Yes	1/8
2	No	Yes	81	Yes	1/8
3	Yes	No	95	Yes	1/8
4	Yes	Yes	75	Yes	1/8
5	No	Yes	70	No	1/8
6	No	Yes	56	No	1/8
7	Yes	No	76	No	1/8
8	Yes	Yes	78	No	1/8

At the start, all samples get the same weight

$$\text{weight of each sample} = \frac{1}{\text{Number of samples}}$$

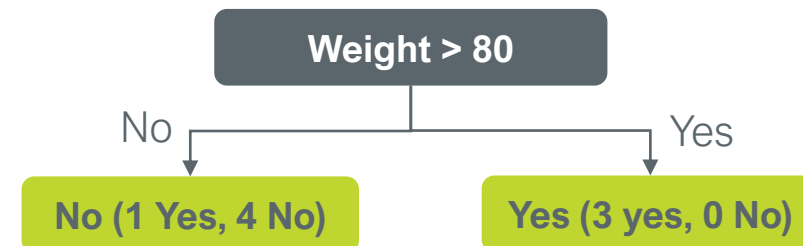
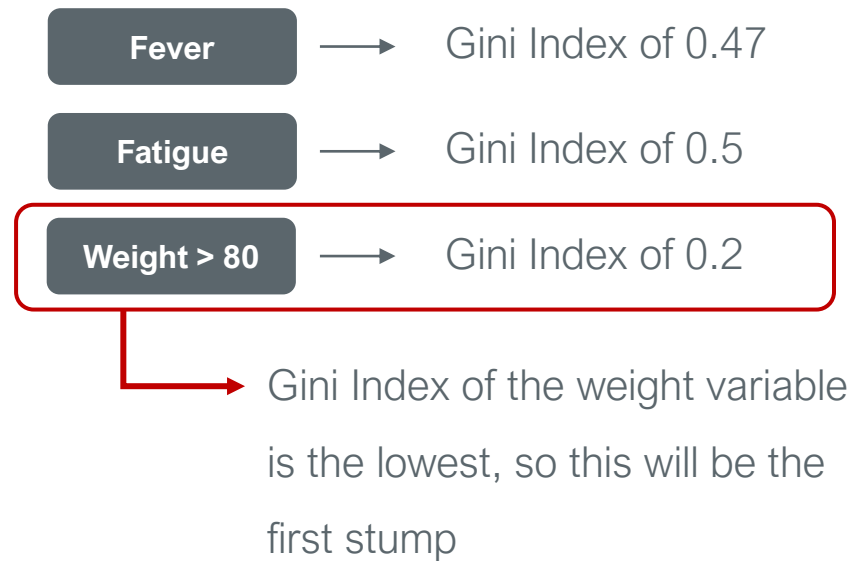
All samples are equally important!

2.1.2. Adaboost

The process

2nd Step: Create the first stump by selecting the variable that best classifies the samples

ID	Fever	Fatigue	Weight	Disease	Sample Weight
1	Yes	Yes	92	Yes	1/8
2	No	Yes	81	Yes	1/8
3	Yes	No	95	Yes	1/8
4	Yes	Yes	75	Yes	1/8
5	No	Yes	70	No	1/8
6	No	Yes	56	No	1/8
7	Yes	No	76	No	1/8
8	Yes	Yes	78	No	1/8

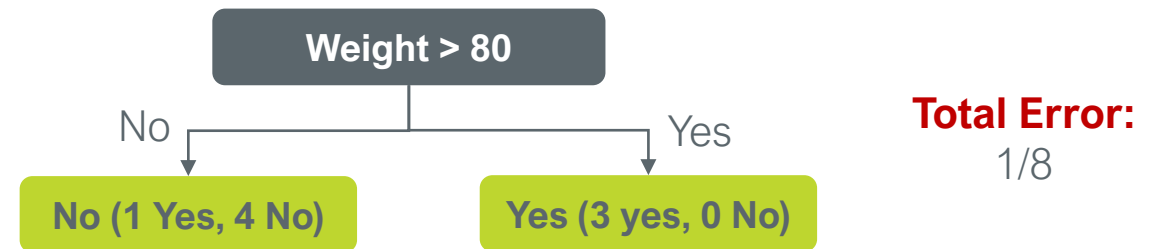


2.1.2. Adaboost

The process

3rd Step: Calculate the total error and define the amount of say (the weight) that this stump will have on the final prediction

ID	Fever	Fatigue	Weight	Disease	Sample Weight
1	Yes	Yes	92	Yes	1/8
2	No	Yes	81	Yes	1/8
3	Yes	No	95	Yes	1/8
4	Yes	Yes	75	Yes	1/8
5	No	Yes	70	No	1/8
6	No	Yes	56	No	1/8
7	Yes	No	76	No	1/8
8	Yes	Yes	78	No	1/8



$$\text{Amount of say} = \frac{1}{2} \log \left(\frac{1 - \text{Total Error}}{\text{Total Error}} \right)$$

$$\text{Amount of say} = \frac{1}{2} \log \left(\frac{1 - 1/8}{1/8} \right) = 0.97$$

Note: The higher the error, the lower the amount of say of that stump

2.1.2. Adaboost

The process

4th Step: Modify the weights so the next stump will consider the errors that the current stump made

ID	Sample Weight	New Sample Weight	Norm. Sample Weight
1	1/8	0.05	0.07
2	1/8	0.05	0.07
3	1/8	0.05	0.07
4	1/8	0.33	0.49
5	1/8	0.05	0.07
6	1/8	0.05	0.07
7	1/8	0.05	0.07
8	1/8	0.05	0.07

We need to emphasize the weight of this sample for the next stump to correctly classify it by increasing its sample weight...

$$\text{New sample weight} = \text{sample weight} \times e^{\text{Amount of say}}$$

$$\text{New sample weight} = \frac{1}{8} \times e^{0.97} = 0.33$$

And decrease the remaining ones...

$$\text{New sample weight} = \text{sample weight} \times e^{-\text{Amount of say}}$$

$$\text{New sample weight} = \frac{1}{8} \times e^{-0.97} = 0.05$$

And finally normalize the weights so the sum is equal to 1

2.1.2. Adaboost

The process

5th Step: Go to step 2 to create a second stump, but this time calculate Weighted Gini Indexes to determine which variable should split the next stump.

ID	Fever	Fatigue	Weight	Disease	Norm. Sample Weight
1	Yes	Yes	92	Yes	0.07
2	No	Yes	81	Yes	0.07
3	Yes	No	95	Yes	0.07
4	Yes	Yes	75	Yes	0.49
5	No	Yes	70	No	0.07
6	No	Yes	56	No	0.07
7	Yes	No	76	No	0.07
8	Yes	Yes	78	No	0.07

→ This new weights will affect the way that Gini index is calculated for each variable in the following step

The Gini Index from previous class...

$$gini(c) = 1 - \sum_{i=1}^n p_i^2$$

Weighted Gini Index p_i :

$$p_i = \frac{w_i \times n_i}{w_i \times n_i (1 - w_i)(1 - n_i)}$$

Where n_i is the number of observations of class i in c , and w_i is the weights assigned to class i

2.1.3. Adaboost

On new predictions

ID	Fever	Fatigue	Weight	Disease
20	Yes	Yes	65	?

How a Forest of Stumps created by Adaboost makes new classifications?

The stumps that indicate that the new patient **has the disease**.



Total Amount of Say: $0.86 + 0.44 + 0.78 = 2.08$

The stumps that indicate that the new patient **do not has the disease**.

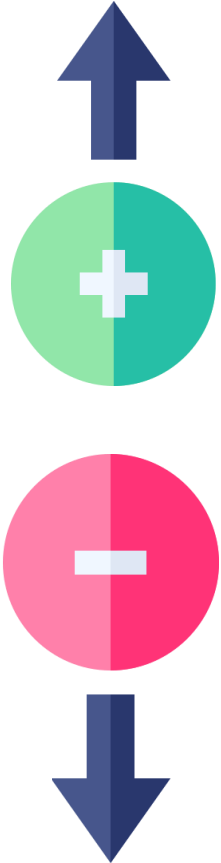


Total Amount of Say: $0.41 + 0.82 = 1.23$

Conclusion: The value 2.08 is higher than 1.23, so the new patient has the disease

2.1.4. Adaboost

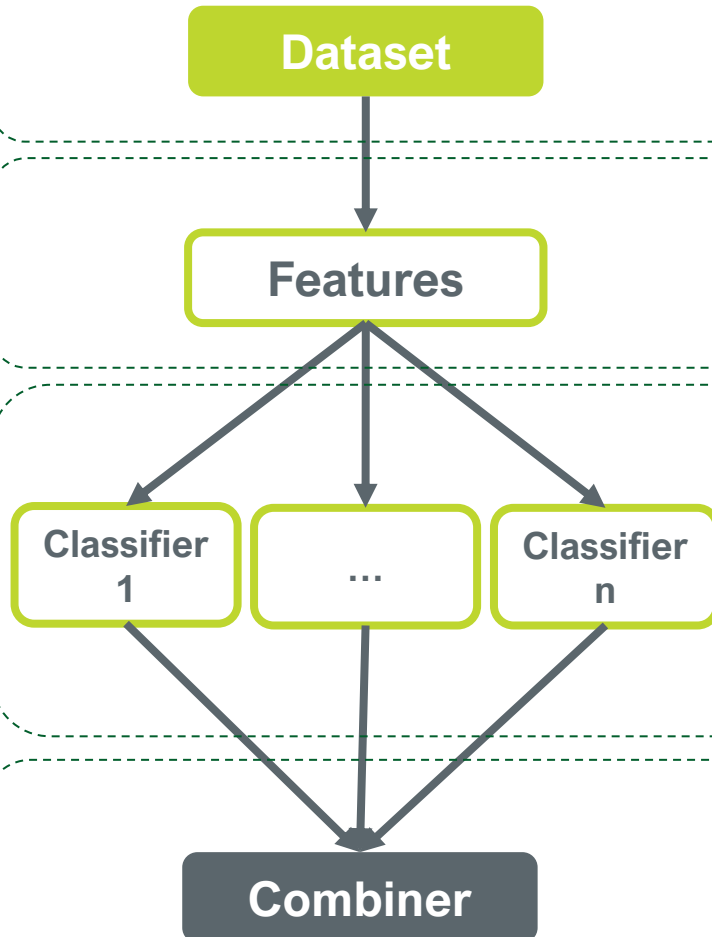
Advantages and Disadvantages



- **Fast** and **simple** to program
 - Has a **small number** of **hyperparameters** to adjust
 - In theory, works with any learning algorithm and is compatible with multiple forms and types of data
 - Will **find a good solution** for many complex problems
-
- Depends on data and weak learners
 - **Prone to overfit** if weak learners are too complex and to underfit if weak learners are too weak
 - Empirically **sensible to noise**

2.1. Adaboost

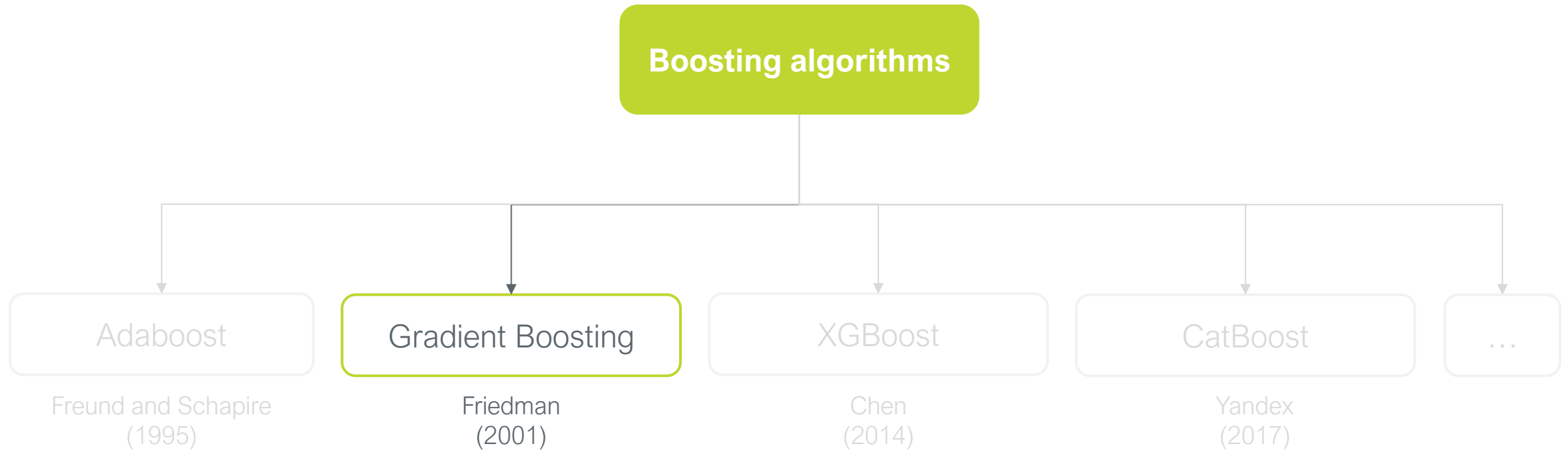
RECAP



- Uses **all training set** observations but **reweights** the observations in each iteration, giving more weight to misclassified instances.
- Uses **all features** in the dataset
- Uses a weak learner as its base classifier (usually a stump)
- The training is **sequential** (each subsequent model trained to correct the errors of the previous models)
- **Regression:** A weighted sum of model outputs
- **Classification:** Weighted voting based on model accuracy (usually soft voting)

2.2. Gradient Boosting

RECAP



2.2.1. Gradient Boosting

Introduction

Since both are boosting methods, AdaBoost and Gradient Boosting have a similar workflow.

What are the differences?

1. The trees

Gradient Boosting uses trees larger (usually ranging from 8 to 32 leaves) than a Decision Stump.

2. The Learning

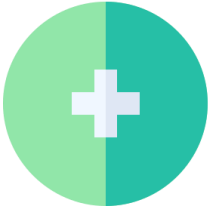
Instead of using sample weight to guide the next Decision Stump, Gradient Boosting uses the residual made by the Decision Tree to guide the next tree.

3. The final prediction

While in Adaboost each classifier has different weights assigned to the final prediction based on its performance, in Gradient Boosting all classifiers are weighed equally and their predictive capacity is restricted with learning rate to increase accuracy.

2.2.2. Gradient Boosting

Advantages and Disadvantages compared to Adaboost



Higher Predictive Accuracy

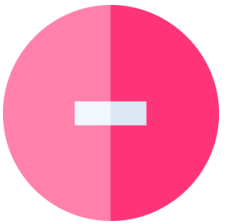
- Usually associated with higher predictive accuracy when compared to Adaboost

Flexibility in Model Complexity and Performance Tuning

- Gradient Boosting allows for greater flexibility in model complexity and performance tuning than Adaboost

Handling Noisy Data

- Gradient Boosting can handle noisy data more effectively compared to AdaBoost



Training Time

- Gradient Boosting usually requires more iterations than Adaboost (slower training)

Prone to Overfitting

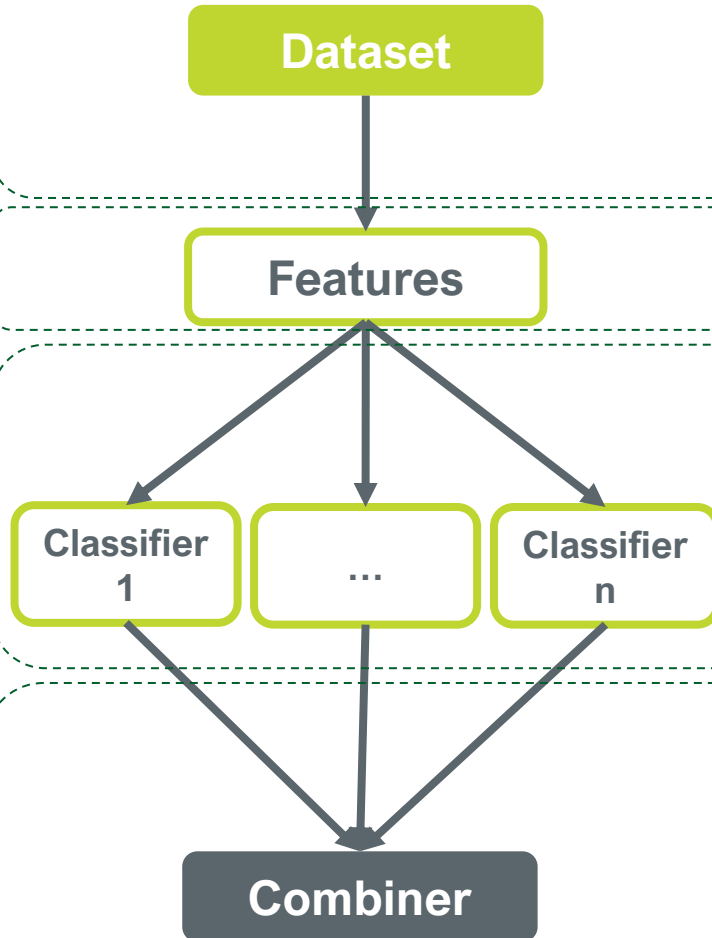
- Gradient Boosting is more prone to overfitting than Adaboost, due to its higher model complexity

Implementation Complexity

- The implementation is more complex than in the Adaboost due to the tuning of hyperparameters

2.2. Gradient Boosting

RECAP



- Uses **all training set** observations but assigns **varying weights** based on residuals, prioritizing instances with higher errors in subsequent iterations.
- Uses **all features** in the dataset
- Uses **decision trees** as its base learners
- The training is **sequential** (each subsequent model trained to minimize the errors made by the ensemble of models built so far)
- Typically, a large number of trees are used
- **Regression:** Final prediction is the sum of individual model predictions
- **Classification:** Final prediction is obtained by summing model outputs to form a score, converting it into probabilities using sigmoid (binary) or softmax (multiclass), and selecting the class with the highest probability.



03.

Stacking Ensemble

3. Stacking Ensemble

Introduction



**What is
stacking?**

Ensemble learning method that combines the predictions
of **multiple base models** using a **meta learner**
to get a final prediction

3.1. Stacking Ensemble

Introduction

Stacking ensembles

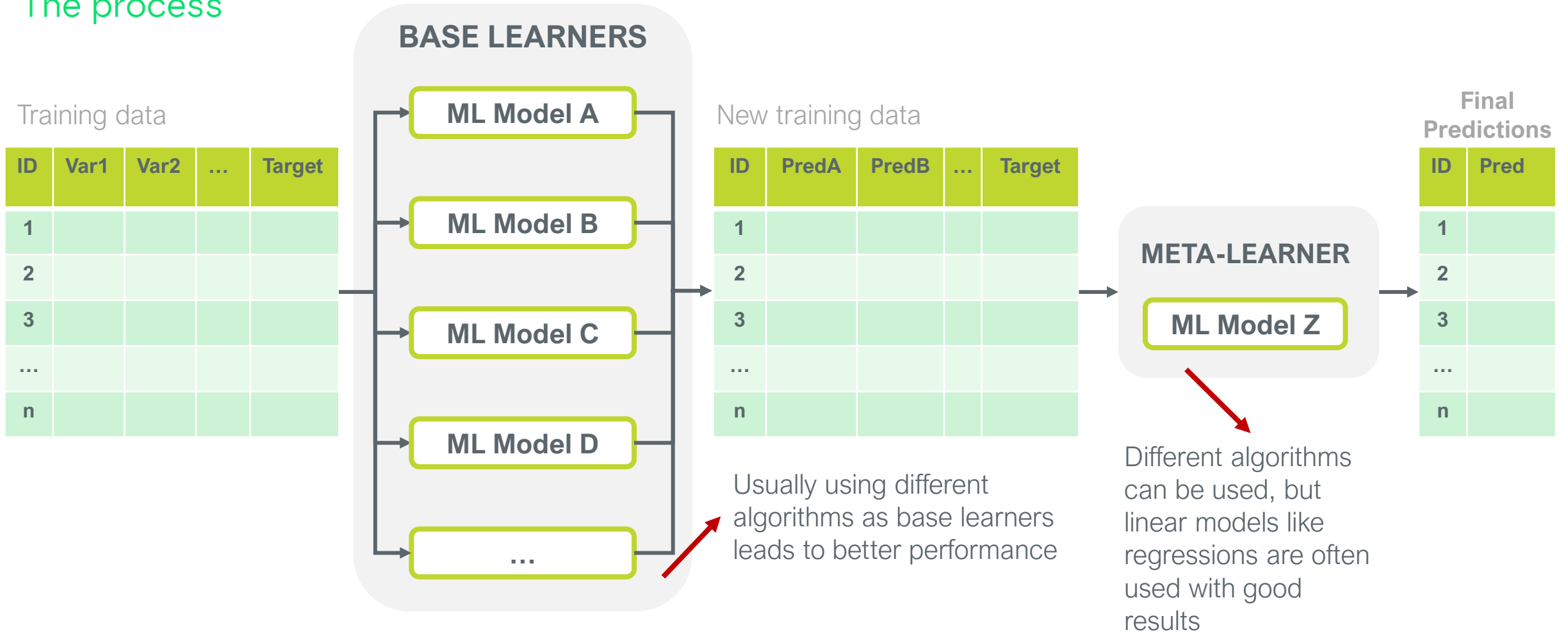
- Use a **heterogeneous set of different algorithms** as base learners
 - Bagging and Boosting consider homogeneous weak learners
- Use a **Meta-Learner** to combine the outputs of the base learners
- Also called blending

Levels of Models

- **Base Learners / Level-0 Learners:** learners that take data as its inputs and generate predictions from it
- **Meta Learners / Level-1 Learners:** learners that take as input the outputs of learners at lower levels

3.2. Stacking Ensemble

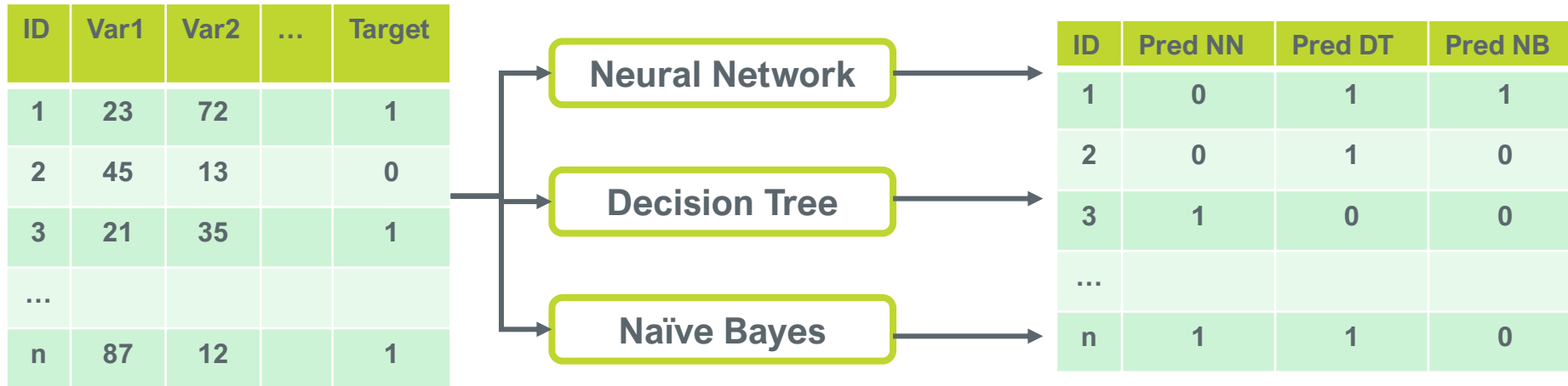
The process



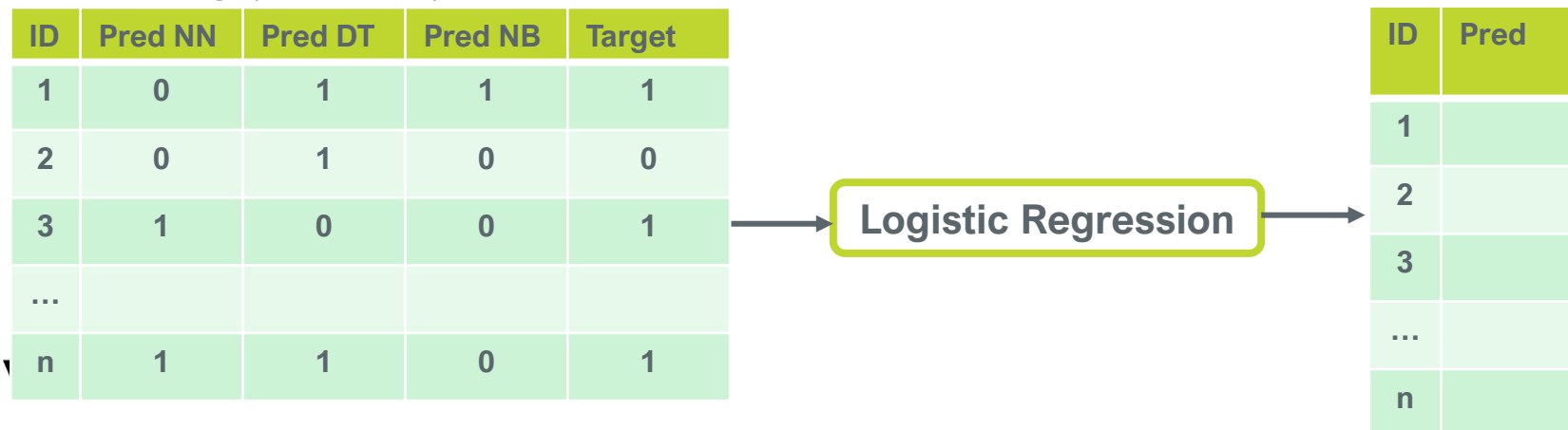
3.3. Stacking Ensemble

Example

On training (1st Level)...



On training (2nd Level)...

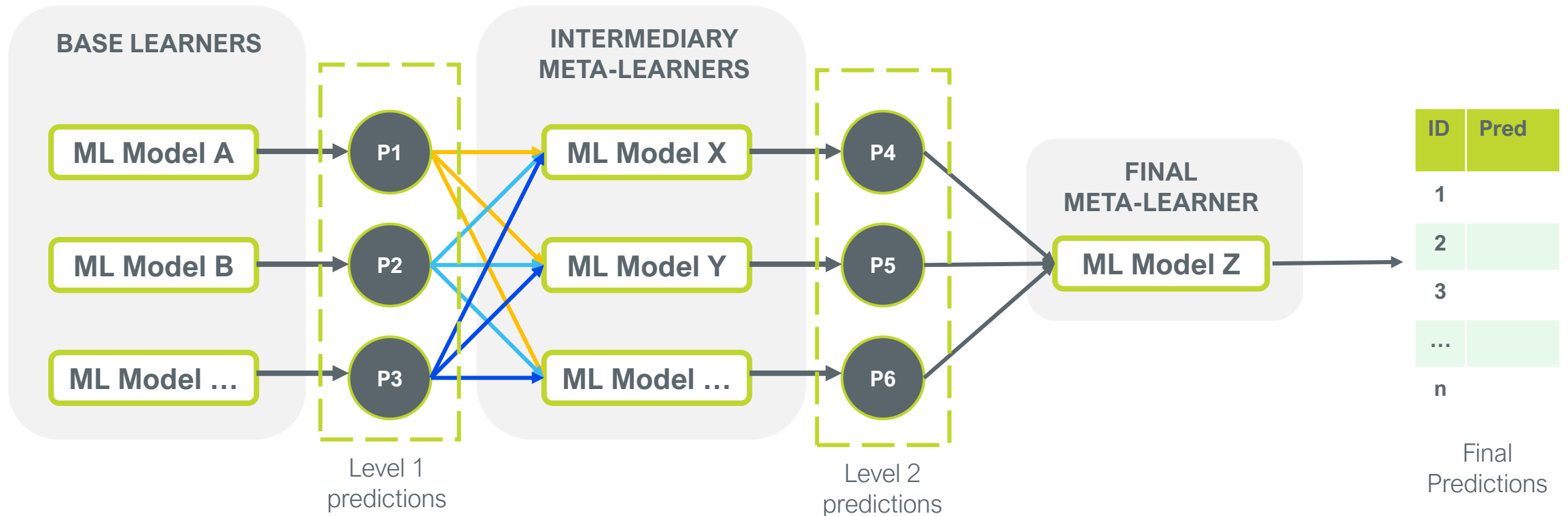


3.4. Stacking Ensemble

Multi-level Stacking

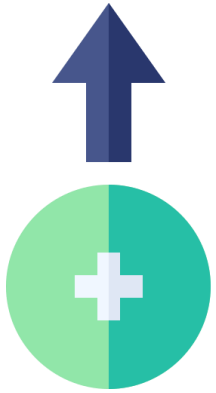
Going further...

Stacking ensembles can have multiple layers!

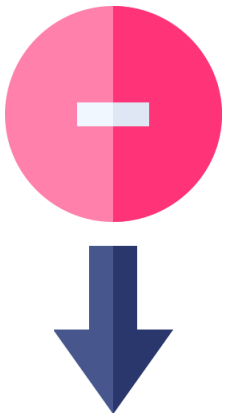


3.5. Stacking Ensemble

Advantages and Disadvantages



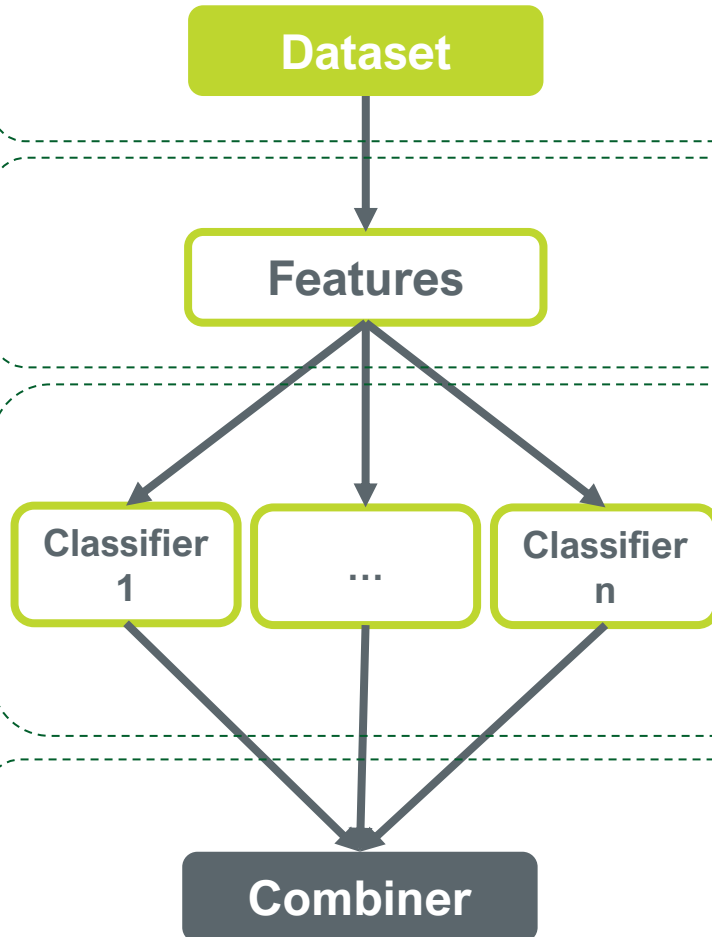
- Often leads to **better predictive performance** by combining different models that capture distinct patterns
 - This can lead to a more robust and generalized model
- The **flexibility** on the choice of models (to use as base-learners and meta-learners) compared to other ensemble techniques
- By combining different strengths of different algorithms, it **may be able to capture complex patterns** that may be difficult for an individual model to learn



- More **computationally expensive**
- Prone to **overfitting**
- **Increased training time** by adding the train of multiple base-models with the train of a meta-model
- Increased time spent on **hyperparameter adjustment on the different models**

3. Stacking Ensemble

RECAP



- Does not modify the training set directly but instead uses the predictions of base learners as additional features for training the meta-learner.
- In the **base learners**, uses the **original features**
- In the **meta-learner**, uses the **predictions** from the base models as input features
- Can use a **variety of base models**, including different algorithms or variations of the same algorithm with different hyperparameters.
- It's common to use multiple diverse base models to capture different aspects of the data.
- The final prediction is **made by the meta-model** using techniques like linear regression, logistic regression or another ML algorithm.

References

- Dietterich, Thomas G. "Ensemble methods in machine learning." Multiple Classifier Systems: First International Workshop, MCS 2000 Cagliari, Italy, June 21–23, 2000 Proceedings 1. Springer Berlin Heidelberg, 2000.
- Bishop, Christopher M., and Nasser M. Nasrabadi. Pattern recognition and machine learning. Vol. 4. No. 4. New York: springer, 2006.
- Breiman, Leo. "Bagging predictors." Machine learning 24 (1996): 123-140.
- <https://towardsdatascience.com/ensemble-methods-bagging-boosting-and-stacking-c9214a10a205>
- <https://cs229.stanford.edu/notes2021fall/section8notes-ensembling-techniques.pdf>
- <https://www.cs.princeton.edu/courses/archive/spring12/cos598A/slides/intro.pdf>

Obrigada!

Thank you!

Acreditações e Certificações da NOVA IMS



Cofinanciado por

